

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

6

PHASE SIX

Advanced FastAPI Features

Move from working to professional: routers, middleware, background tasks, file upload/download, WebSockets, lifespan events, settings via pydantic-settings, and standardised pagination, filtering, and sorting.

Contents

6.1 API Routers (APIRouter)	4
Why split into routers?	4
Router-level dependencies	5
6.2 Middleware	6
Built-in middleware	6
Custom middleware: a request timer	6
Order of execution	7
6.3 Background Tasks	7
6.4 File Upload and Download	8
Receiving uploads	8
Sending downloads	9
6.5 WebSockets (Brief Introduction)	9
6.6 Lifespan Events (Startup & Shutdown)	10
6.7 Environment Configuration with Pydantic Settings	11
6.8 Pagination, Filtering, Sorting Patterns	13
Pagination	13
Filtering and sorting	13
Hands-On Exercise 6: Refactor the Book Library	14
Checkpoint: At This Point You Can Now...	17
Phase 6 Summary	18

Your Journey

✓	Foundations: What Is an API?	DONE
✓	Setup & Your First API	DONE
✓	Path Params, Query Params & Body	DONE
✓	Pydantic: Data Validation	DONE
✓	Database Integration (SQLAlchemy)	DONE
✓	Authentication & Authorization	DONE
6	Advanced FastAPI Features	YOU ARE HERE
7	Testing Your FastAPI App	
8	Deployment with Docker	
★	Capstone — TaskFlow API	

Your API now works and is secure, but it's all crammed into a growing pile of files. This phase is about the patterns that keep an app maintainable as it scales — and a few powerful features (background tasks, WebSockets, lifespan events) you'll want in your toolkit. None of these are hard individually; together they're what separates a tutorial project from a production codebase. Let's level up.

6.1 API Routers (`APIRouter`)

Why split into routers?

So far every endpoint lives on the single `app` object. As an app grows to dozens of endpoints across users, books, auth, and admin, one giant file becomes unmaintainable. `APIRouter` lets you group related endpoints into separate files (a "mini-app" each), then plug them into the main app. It's the FastAPI equivalent of organising code into modules.

PYTHON

```
# app/routers/books.py
from fastapi import APIRouter

# prefix: every route here starts with /books.
# tags: groups these endpoints together in the /docs UI.
router = APIRouter(prefix="/books", tags=["books"])

@router.get("") # becomes GET /books
def list_books():
    return []

@router.get("/{book_id}") # becomes GET /books/{book_id}
def get_book(book_id: int):
    return {"book_id": book_id}
```

Then include each router in `main.py`:

PYTHON

```
# app/main.py
from fastapi import FastAPI
from app.routers import books, users, auth

app = FastAPI(title="Book Library API")

# Mount each router; their prefixes and tags apply automatically.
app.include_router(auth.router)
app.include_router(users.router)
app.include_router(books.router)
```

Router-level dependencies

You can attach a dependency to an *entire* router, so it runs for every route inside — perfect for an admin section where every endpoint requires admin rights:

PYTHON

```
from fastapi import APIRouter, Depends

# Every route in this router will require the admin check.
admin_router = APIRouter(
    prefix="/admin",
    tags=["admin"],
    dependencies=[Depends(require_admin)],
)
```

TIP

Tip: Organise routers by **resource/domain** — one file per entity (`books.py` , `users.py` , `tags.py`). This is exactly the structure you'll use in the capstone, where each entity gets its own router file. It keeps related logic together and makes the codebase navigable.

CONCEPT CHECK

Concept Check: If a router has `prefix="/books"` and a route decorated `@router.get("/{book_id}")` , what's the full path? Answer: `/books/{book_id}` . The router prefix is prepended to each route's own path.

6.2 Middleware

Middleware is code that runs for every request, wrapping around your route handlers — like a series of checkpoints a request passes through on the way in and the response passes through on the way out. Common uses: logging, timing, adding headers, compression.

TEXT

Request → [Middleware A → Middleware B → route handler] → Middleware B → Middleware A → Response

Built-in middleware

FastAPI (via Starlette) ships several useful ones:

- `CORSMiddleware` — controls cross-origin browser access (you met this in Phase 5).
- `GZipMiddleware` — compresses responses to save bandwidth.
- `TrustedHostMiddleware` — rejects requests with unexpected `Host` headers.

Custom middleware: a request timer

You can write your own. This one measures how long each request takes and adds the result as a response header:

PYTHON

```
import time
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def add_timing_header(request: Request, call_next):
    start = time.perf_counter()      # before the request is handled
    response = await call_next(request) # run the rest of the app
    duration = time.perf_counter() - start # after we have the response
    # Attach the timing as a custom header on the way out.
    response.headers["X-Process-Time"] = f"{duration:.4f}"
    print(f"{request.method} {request.url.path} took {duration:.4f}s")
    return response
```

The `call_next(request)` call is the pivot: everything *before* it runs on the way in, `call_next` runs the actual route, and everything *after* runs on the way out with the response in hand.

Order of execution

Middleware runs in **reverse order of registration** for the request, and forward order for the response — the last one added is the outermost layer. If order matters (e.g. you want CORS to wrap everything), mind the sequence in which you add them.

COMMON MISTAKES

Common Mistakes: Doing heavy or blocking work inside middleware. Because it runs on every request, a slow middleware slows your entire API. Keep middleware light; push real work into the route or a background task.

6.3 Background Tasks

Some work shouldn't make the client wait — sending a welcome email, writing an audit log, generating a thumbnail. **Background tasks** let you return the response *immediately* and run that work *afterward*.

PYTHON

```
from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

def write_log(message: str):
    # This runs AFTER the response is sent – the client doesn't wait for it.
    with open("log.txt", "a") as f:
        f.write(message + "\n")

@app.post("/users")
def create_user(email: str, background_tasks: BackgroundTasks):
    # ... create the user ...
    # Schedule the slow work; the response returns right away.
    background_tasks.add_task(write_log, f"welcome email queued for {email}")
    return {"message": "User created"}
```

You declare a `BackgroundTasks` parameter, then call `add_task(function, *args)`. FastAPI sends the response first, then runs the queued tasks.

TIP

Tip — background tasks vs `async` : They solve different problems. `async / await` lets the server handle *other requests* while waiting on I/O *during* a request. Background tasks run work *after the response is already sent* to the current client. Use background tasks when the client doesn't need to wait for the result (fire-and-forget).

WARNING

Warning: Background tasks run in the *same process* as your app, so they're great for light work (logging, a quick email) but not for heavy, long-running jobs (video encoding, big reports). For those, use a dedicated task queue like **Celery** (mentioned in "What to Learn Next"). Background tasks are a scalpel, not a bulldozer.

6.4 File Upload and Download

Receiving uploads

Use `UploadFile` to accept file uploads. It streams the file (so large files don't exhaust memory) and exposes metadata:

PYTHON

```
from fastapi import FastAPI, UploadFile, File, HTTPException

app = FastAPI()

@app.post("/upload")
async def upload(file: UploadFile = File(...)):
    # Validate the type before trusting it.
    if not file.content_type.startswith("image/"):
        raise HTTPException(400, "Only images are allowed")
    # Read the bytes and save to disk.
    contents = await file.read()
    with open(f"uploads/{file.filename}", "wb") as f:
        f.write(contents)
    return {"filename": file.filename, "size": len(contents),
            "type": file.content_type}
```

COMMON MISTAKES

Common Mistakes: Trusting an uploaded file's name or type blindly. A malicious client can send any `filename` (including path tricks like `../../etc/passwd`) and lie about `content_type`. Always validate the type, sanitise or generate your own filename, and enforce a size limit.

Sending downloads

Use `FileResponse` to serve a file from disk (you saw this in Phase 2):

PYTHON

```
from fastapi.responses import FileResponse

@app.get("/download/{filename}")
def download(filename: str):
    return FileResponse(f"uploads/{filename}")
```

For very large or generated-on-the-fly content, `StreamingResponse` sends it in chunks without loading it all into memory.

6.5 WebSockets (Brief Introduction)

Everything so far has been **HTTP**: a request gets one response, then the connection closes. **WebSockets** are different — they open a *persistent, two-way* connection so the

server and client can send messages to each other at any time. Think of HTTP as sending letters and WebSockets as an open phone call.

PYTHON

```
from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()          # open the connection
    while True:
        # Wait for a message, then echo it back. Stays open between messages.
        data = await websocket.receive_text()
        await websocket.send_text(f"You said: {data}")
```

Use cases: live chat, real-time dashboards, collaborative editing, live notifications — anything where the server needs to *push* updates without the client asking. (A WebSocket live-task-feed is a capstone bonus.)

CONCEPT CHECK

Concept Check: A stock-price ticker that updates the page the instant a price changes — HTTP polling or WebSocket? *Answer: WebSocket. The server pushes each change over the open connection instantly, instead of the client repeatedly asking "any updates yet?" over many HTTP requests.*

6.6 Lifespan Events (Startup & Shutdown)

Sometimes you need to run code once when the app **starts** (connect to a database, load a machine-learning model into memory, warm a cache) and once when it **shuts down** (close connections, flush logs). The modern way is a `Lifespan` context manager:

PYTHON

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # --- startup: runs once before the app accepts requests ---
    print("Starting up: connecting resources, loading models...")
    ml_model = load_model()
    app.state.model = ml_model      # store it for routes to use
    yield                          # <-- the app runs while we're "paused" here
    # --- shutdown: runs once as the app stops ---
    print("Shutting down: closing connections...")
    ml_model.cleanup()

# Hand the lifespan to the app.
app = FastAPI(lifespan=lifespan)
```

The single `yield` splits the function: everything **above** runs at startup, everything **below** runs at shutdown, and the app serves requests in between.

TIP

Tip: A common pattern is loading something expensive *once* at startup (an ML model, a big config file, a connection pool) and stashing it on `app.state`, rather than reloading it on every request. Startup cost is paid once; every request after is fast.

6.7 Environment Configuration with Pydantic Settings

Hardcoding values like `SECRET_KEY` or `DATABASE_URL` in your source is insecure and inflexible — different environments (dev, staging, production) need different values, and secrets must never be committed to git. The clean solution is `pydantic-settings`, which reads configuration from environment variables and a `.env` file, *with validation*.

BASH / TERMINAL

```
pip install pydantic-settings
```

PYTHON

```
# app/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    # These are read from environment variables or the .env file,
    # and validated/typed just like any Pydantic model.
    database_url: str
    secret_key: str
    access_token_expire_minutes: int = 15

    # Tell pydantic-settings which file to read.
    model_config = SettingsConfigDict(env_file=".env")

# Create one instance to import everywhere.
settings = Settings()
```

Your `.env` file holds the actual values (and is **never** committed):

TEXT

```
DATABASE_URL=postgresql://user:pass@localhost:5432/booklib
SECRET_KEY=a-very-long-random-secret-string
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

And you commit a `.env.example` (with placeholder values) so teammates know what's required:

TEXT

```
DATABASE_URL=
SECRET_KEY=
ACCESS_TOKEN_EXPIRE_MINUTES=15
```

Now anywhere in your code, `from app.config import settings` and use `settings.secret_key`. If a required variable is missing, the app fails loudly at startup — far better than a confusing error deep in a request.

WARNING

Warning: Add `.env` to your `.gitignore` immediately. Committing real secrets to a repository — even a private one — is one of the most common and damaging security mistakes. Commit `.env.example` (placeholders only); never `.env`.

CONCEPT CHECK

Concept Check: Why commit `.env.example` but not `.env`? Answer: `.env.example` documents which settings exist (with safe placeholder values) so teammates can set up their own. `.env` contains the real secrets, which must never enter version control.

6.8 Pagination, Filtering, Sorting Patterns

List endpoints should never return *everything* — a table with a million rows would crash the client and your server. Standardise three things across all your list endpoints:

Pagination

Return a consistent envelope with the items plus metadata, using `page` / `page_size` query parameters:

PYTHON

```
from pydantic import BaseModel

class PaginatedBooks(BaseModel):
    items: list[BookResponse]
    total: int          # total matching rows (before paging)
    page: int
    page_size: int

@app.get("/books", response_model=PaginatedBooks)
def list_books(page: int = 1, page_size: int = 20,
               db: Session = Depends(get_db)):
    query = db.query(Book)
    total = query.count()
    # offset skips earlier pages; limit caps this page's size.
    items = query.offset((page - 1) * page_size).limit(page_size).all()
    return {"items": items, "total": total,
            "page": page, "page_size": page_size}
```

Filtering and sorting

Accept optional query parameters and apply them conditionally:

PYTHON

```
@app.get("/books2")
def list_books2(genre: str | None = None, sort_by: str = "title",
                order: str = "asc", db: Session = Depends(get_db)):
    query = db.query(Book)
    if genre:                                     # filter only if provided
        query = query.filter(Book.genre == genre)
    column = getattr(Book, sort_by, Book.title) # pick the sort column safely
    query = query.order_by(column.desc() if order == "desc" else column.asc())
    return query.all()
```

TIP

Tip: Always set a sensible **maximum** `page_size` (e.g. cap it at 100), so a client can't request a million rows in one go and overwhelm your server. `page_size: int = Query(default=20, le=100)` enforces this automatically.

HANDS-ON

Hands-On Exercise 6: Refactor the Book Library

Bring it all together. Refactor the Book Library to use separate routers, a request-timing middleware, a background task that logs each book creation to a file, settings from `.env`, and a paginated `GET /books`. Below is a compact single-file version demonstrating each piece working together (in a real project these live in separate files per 4.2 / 6.1).

```

# advanced_app.py
import time
from contextlib import asynccontextmanager
from fastapi import (FastAPI, APIRouter, Request, BackgroundTasks,
                     Depends, Query)
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict

# ---- settings from .env (with defaults so it runs out of the box) ----
class Settings(BaseSettings):
    app_name: str = "Book Library API"
    log_file: str = "book_events.log"
    model_config = SettingsConfigDict(env_file=".env")

settings = Settings()

# ---- lifespan: startup/shutdown ----
@asynccontextmanager
async def lifespan(app: FastAPI):
    print(f"Starting {settings.app_name}")
    app.state.books = []          # our in-memory store for the demo
    app.state.next_id = 1
    yield
    print("Shutting down")

app = FastAPI(title=settings.app_name, lifespan=lifespan)

# ---- middleware: time every request ----
@app.middleware("http")
async def timing(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time"] = f"{time.perf_counter() - start:.4f}"
    return response

# ---- background task ----
def log_book_event(title: str):
    with open(settings.log_file, "a") as f:
        f.write(f"created: {title}\n")

# ---- schemas ----
class BookCreate(BaseModel):
    title: str = Field(min_length=1)
    genre: str = "general"

class BookResponse(BookCreate):
    id: int

class PaginatedBooks(BaseModel):
    items: list[BookResponse]

```

```

total: int
page: int
page_size: int

# ---- router ----
router = APIRouter(prefix="/books", tags=["books"])

@router.post("", response_model=BookResponse, status_code=201)
def create_book(payload: BookCreate, bg: BackgroundTasks, request: Request):
    store = request.app.state
    book = BookResponse(id=store.next_id, **payload.model_dump())
    store.books.append(book)
    store.next_id += 1
    # Log AFTER responding – client doesn't wait.
    bg.add_task(log_book_event, book.title)
    return book

@router.get("", response_model=PaginatedBooks)
def list_books(request: Request, page: int = 1,
               page_size: int = Query(default=20, le=100),
               genre: str | None = None):
    books = request.app.state.books
    if genre:
        books = [b for b in books if b.genre == genre]
    total = len(books)
    start = (page - 1) * page_size
    return {"items": books[start:start + page_size], "total": total,
            "page": page, "page_size": page_size}

app.include_router(router)

```

Run and test:

BASH / TERMINAL

```
uvicorn advanced_app:app --reload
```

TEXT

```

POST /books {"title":"Dune","genre":"scifi"} → 201; check response header X-Process-Time
POST /books {"title":"1984","genre":"scifi"} → 201
GET  /books?page=1&page_size=1                → {"items":[Dune], total:2, page:1,
page_size:1}
GET  /books?genre=scifi                       → both books
GET  /books?page_size=999                     → 422 (capped at 100)
# After the POSTs, book_events.log contains the created titles.

```

What just happened? You used six production patterns in one small app. The endpoints live on an **APIRouter** (so they'd sit in their own file). A **middleware** times every request and stamps **X-Process-Time** on the response. A **background task** logs each creation to a

file *after* the response is sent, so the client never waits. `pydantic-settings` supplies configuration (with `.env` support). A **lifespan** handler sets up and tears down app state. And **pagination** with a capped `page_size` plus optional **filtering** keeps the list endpoint safe and predictable. This is what a real FastAPI codebase looks like — and exactly the toolkit you'll assemble into the capstone.

COMMON MISTAKES

Common Mistakes: Putting business logic in middleware or forgetting to cap `page_size`. Middleware should be light and generic; per-resource logic belongs in routes. And an uncapped `page_size` is a denial-of-service waiting to happen — always bound it with `Query(le=...)`.

TIP

Tip: Notice how cleanly these compose: routers organise, middleware observes, background tasks defer, settings configure, lifespan manages resources, and pagination protects. Each concern is isolated. That isolation is precisely what makes large codebases maintainable.

Checkpoint: At This Point You Can Now...

- ☐ Split endpoints into `APIRouter` files with `prefix`, `tags`, and router-level dependencies, and mount them with `include_router`.
- ☐ Write **custom middleware** and explain `call_next` and execution order.
- ☐ Defer work with **background tasks** and distinguish them from `async`.
- ☐ Handle **file uploads** (`UploadFile` / `File`) safely and serve **downloads** (`FileResponse`).
- ☐ Explain what **WebSockets** are and when to use them over HTTP.
- ☐ Run startup/shutdown logic with a **lifespan** context manager.
- ☐ Load validated configuration with `pydantic-settings` and a `.env` file (and know to git-ignore it).
- ☐ Implement standardised **pagination, filtering, and sorting** with a capped page size.

Phase 6 Summary

PHASE 6 • KEY TAKEAWAYS

In this phase you adopted the patterns that make a FastAPI app production-ready and maintainable.

- **APIRouter** groups related endpoints into per-resource files with shared **prefix**, **tags**, and dependencies; **include_router** mounts them.
- **Middleware** runs on every request around your handlers (**call_next** is the pivot); keep it light. Built-ins cover CORS, GZip, trusted hosts.
- **Background tasks** return the response first and run fire-and-forget work after; different from **async**, and not for heavy jobs (use Celery for those).
- **File uploads** use **UploadFile**/**File** (validate type, size, and filename!); **downloads** use **FileResponse**/**StreamingResponse**.
- **WebSockets** give a persistent two-way connection for real-time push (chat, live dashboards).
- **Lifespan** context managers run startup/shutdown code (load models, open/close connections) with a single **yield** splitting the two.
- **pydantic-settings** loads validated config from env vars and **.env** (commit **.env.example**, never **.env**).
- Standardise **pagination** (items + total + page envelope), **filtering**, and **sorting**, and always **cap** **page_size**.
- **You refactored the Book Library** to use routers, timing middleware, a logging background task, **.env** settings, and paginated listing.

Your app now follows professional structure. Next phase, you'll prove it all works with an automated test suite.

End of Phase 6. Reply "**continue**" for Phase 7 — Testing Your FastAPI App — where you write automated tests with **pytest** and FastAPI's **TestClient**, and measure coverage.